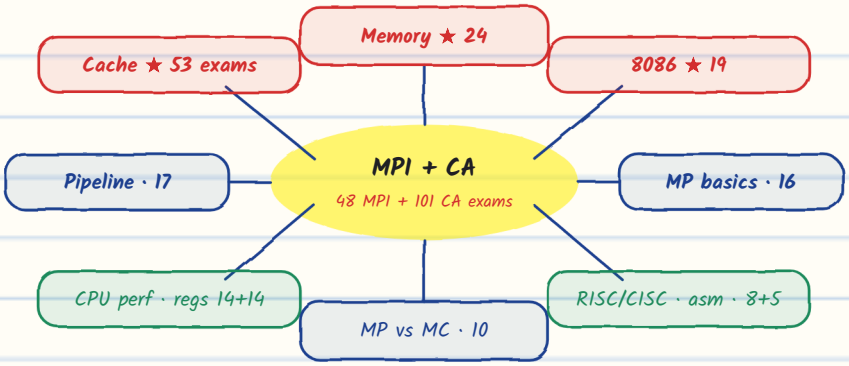
# Microprocessor & Computer Architecture

every 8086, microprocessor, cache, memory and pipeline question asked in BCS · NTRCA · BPSC · bank · power-sector · BUET-set papers, solved in 30 sheets




↑ the number is how many different exams asked that topic (red = most asked)

## ➜ How to read a question card

**Q** The question, as printed in the paper (typos fixed).
**Asked in:** the exams it was taken from, e.g. 45th BCS CSE (2024) · BUET M.Sc. April 2021

No dated exam → the card names its source instead. Then: answer → green = worked result → trap. In MCQ tables the green column is the key.

**Most asked**
① AMAT numericals
② tag / index / offset
③ MP vs microcontroller
④ 8086 physical address
⑤ cache mapping types
⑥ RISC vs CISC, pipeline

## ➜ Syllabus coverage

MPI: "microprocessor vs microcontroller; 8086 architecture, segmentation, flags, addressing modes; assembly; 8255A, interrupts, DMA". CA: "measuring performance; multicycle; hazards; pipeline, superscalar; cache, virtual memory".

**Legend** yellow = key term · red circle = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = worked answer

## ➜ Contents

✓ MP basics, widths … p. 2
✓ MP vs microcontroller … 3
✓ 32/64-bit, MCU design … 4
✓ 8086 blocks, segments … 5–6
✓ flags, banks, 8085 … 7–8
✓ addressing, assembly … 9–11
✓ interrupts, DMA, 8255 … 12, 14
✓ generations, GPU … 13–14
✓ CA basics, RISC/CISC … 15–17
✓ CPU performance … 18
✓ pipelining, hazards … 19–20
✓ memory I–III … 21–23
✓ cache concepts … 24–25
✓ tag/index, AMAT … 26–27
✓ replacement · night before · practice … 28–30

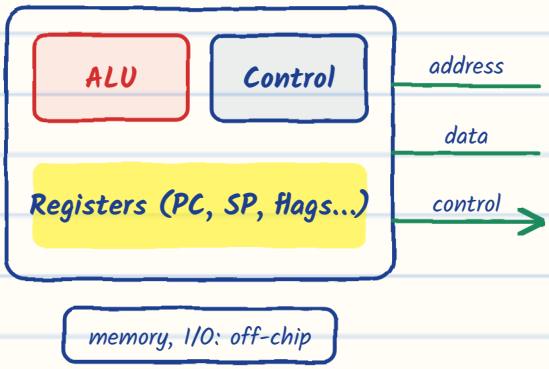
# 1 What a microprocessor is, contains and does

16 exams

**Q1** What is a microprocessor? Explain the basic components of a microprocessor.
✎ **Asked in:** SPCBL AME 2021 · label the parts of a given block diagram: DESCO AE(CSE) 2016

A *microprocessor* is a complete CPU on a single IC: it fetches instructions from memory, decodes and executes them. Memory and I/O are **outside** the chip.

1. **ALU**: arithmetic and logic on data
2. **Control unit**: decodes, sends timing and control signals
3. **Register array**: accumulator, general, PC, SP, flags
4. **Internal buses** + bus interface to address, data and control buses




**Exam trap** A viva answer of "ALU, registers, output circuit" loses marks: the **control unit** must be named and an output circuit is not a CPU part. **Hard disk** is never a component ✎ **Asked in:** BUET IICT April 2021; "built-in memory and I/O" describes a **microcontroller**, not a microprocessor ✎ **Asked in:** BUET M.Sc. April 2021.

**Q2** Write the functions of a microprocessor.
✎ **Asked in:** BTRC DAD 2021
Fetch → decode → execute ✎ **Asked in:** order MCQ: BUET IICT April 2021; move data between memory, registers and I/O; arithmetic and logic; generate timing/control signals; respond to interrupts.

**Q3** Give the data and address widths of an 8-bit and a 16-bit microprocessor.
✎ **Asked in:** PGCB AE(CSE) 2021
8085: data **8**, address **16** → **64 KB**. 8086: data **16**, address **20** → **1 MB**. "n-bit" = width of the data bus/ALU, not the address bus.

| MCQ asked | Key | Asked in |
|---|---|---|
| 16-bit address bus can address at most … KB | 64 KB ($2^{16}$) | ✎ **Asked in:** 49th BCS CSE (2025) |
| 16-bit data bus and 32-bit address bus → max addressing capacity | 4 GB ($2^{32}$) | ✎ **Asked in:** BUET IICT April 2021 |
| The address bus flows … | unidirectional | ✎ **Asked in:** Combined Bank AHE/AE(IT) 2021 |
| Content of the stack pointer (SP) | address of stack top | ✎ **Asked in:** BUET IICT April 2021 |

# 2 Microprocessor vs microcontroller: the most-asked MPI question

10 exams

**Q4** Write the differences between a microprocessor and a microcontroller. (What exactly is a microcontroller? Write 3 differences / hardware-related differences.)

✎ **Asked in:** 46th BCS CSE · 45th BCS ICT (2024) · Parliament Secretariat Programmer 2023 · Combined Bank SO(IT) 2024 · W2PDCL SAE 2023 · SPCBL AME 2021 · NWPGCL JAM 2020 · MM AP 2017 · BUET M.Sc. April 2017

| Point | Microprocessor (8086, Core i5) | Microcontroller (8051, AVR, PIC, ARM Cortex-M) |
|---|---|---|
| **On the chip** | CPU only | CPU + RAM + ROM/Flash + I/O ports + timers + ADC |
| Hardware needed | external memory, I/O chips, buses: big board | almost none: single-chip system |
| Purpose | general purpose (PCs, servers) | one dedicated task (embedded) |
| Speed / power | GHz, tens of watts | MHz, milliwatts; sleep modes |
| Cost / size | high; system is large | low; compact |
| Architecture | mostly Von Neumann | mostly **Harvard** (separate program/data memory) |
| Example use | laptop, desktop | washing machine, microwave, car ECU |

**Q5** Write down the advantages of a microcontroller over a microprocessor.

✎ **Asked in:** BUET M.Sc. October 2015 · BUET M.Sc. May 2016

Fewer chips → cheaper, smaller, more reliable board; low power (battery devices); on-chip timers, ADC, serial ports; fast response to real-time I/O; simpler design and programming for one job.

**Q6** Advantages of an AVR microcontroller over the 80386.

✎ **Asked in:** BUET M.Sc. October 2017

AVR: RISC, mostly 1 instruction per clock, Harvard, on-chip Flash/SRAM/EEPROM and peripherals, milliwatt power. 80386: CISC, needs external memory and support chips.

**Memory trick** "**Processor = brain only; controller = brain + body**". If the question says dedicated / embedded / low power, the answer is the microcontroller.

## 3 32 vs 64 bit, GHz and clock period

bank papers

**Q7** Difference between a 32-bit and a 64-bit microprocessor with example. What is the meaning of a 2.40 GHz microprocessor? (also: what is a 32-bit microprocessor, its key characteristics?)

✎ **Asked in:** Combined Bank SO(IT) 2023 · BUET M.Sc.

| | 32-bit (Pentium 4, ARM Cortex-A7) | 64-bit (Core i7, Ryzen, Apple M-series) |
|---|---|---|
| Register / ALU width | 32 bits per operation | 64 bits per operation |
| Address space | $2^{32}$ = 4 GB | $2^{64}$ in theory (≈ 16 EB); 48-bit in practice |
| OS / software | runs only 32-bit | runs 64-bit and usually 32-bit |

**2.40 GHz** = the clock ticks $2.4 \times 10^9$ times a second; one cycle = 1/2.4 GHz ≈ **0.417 ns**. It is not "instructions per second" (that also depends on CPI and cores). ✎ **Asked in:** time period at 1.8 GHz: 45th BCS CSE (2024) → $T = 1/f = 1/(1.8 \times 10^9)$ = 0.556 ns.

**Embedded system** (bank viva): a computer built into a larger device for one fixed job, usually around a microcontroller, often real-time (ATM, traffic light, pacemaker). ✎ **Source:** Bank IT viva ·

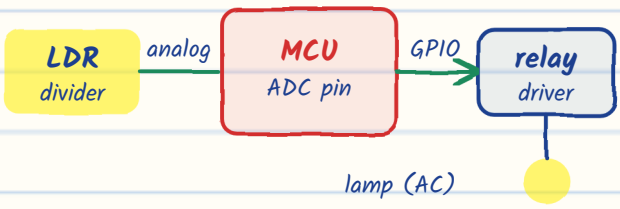
**Microcomputer vs microprocessor**: a microcomputer = microprocessor + memory + I/O on a board (a complete computer) ✎ **Asked in:** 15th NTRCA Computer Operation (2019)

## 4 Design with a microcontroller

new type, 2025

**Q8** Design an automatic street-light controller that turns the lights on when ambient light falls below a threshold.

✎ **Asked in:** DPDC AM(ICT) 2025




```
loop:
  v = ADC_read(LDR)
  if v < T_on  → relay ON
  if v > T_off → relay OFF
  (T_off > T_on: hysteresis)
  sleep 1 s; goto loop
```

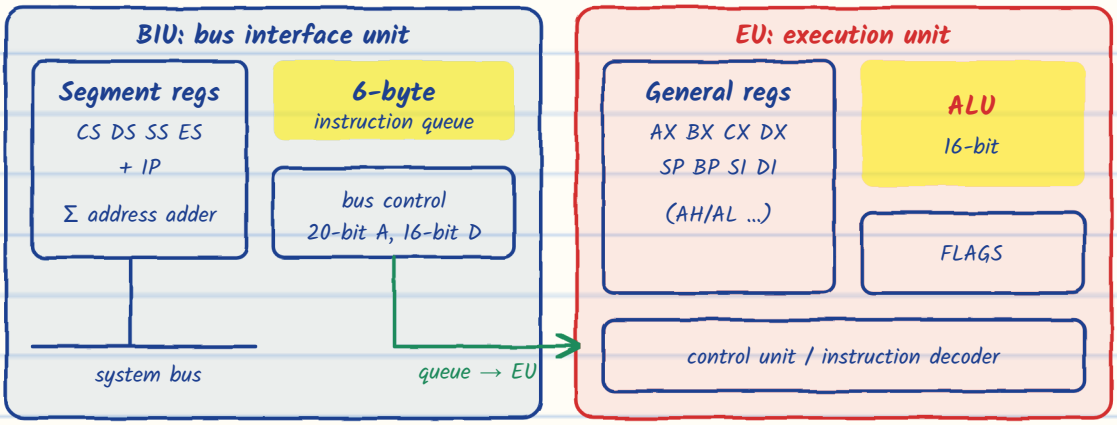
Steps: LDR + resistor as a voltage divider → MCU ADC → compare with a threshold → transistor/relay drives the lamp. **Hysteresis** (two thresholds) stops flicker at dusk; add a time delay so a passing car's headlight is ignored. Same idea for a **gas-leak sensor**: the MQ-module gives an analog voltage (to an ADC pin) and a digital threshold output (high/low, can raise an interrupt). ✎ **Source:** paper not shown on the source sheet

# 5 8086 architecture: BIU + EU

19 exams ✦ top MPI topic

**Q9** Draw the block diagram of the Intel 8086 microprocessor and write its features. (Describe the architecture of 8086.) 5 marks

✎ **Asked in:** 45th BCS CSE (2024) · 36th BCS (2016)




*the BIU fetches ahead into the queue while the EU executes: a 2-stage pipeline*

- ✓ **16-bit** processor: 16-bit ALU, registers, data bus
- ✓ **20-bit** address bus → **1 MB** memory
- ✓ multiplexed AD0–AD15; 40 pins; 5–10 MHz; 1978
- ✓ pipelining by the 6-byte prefetch queue
- ✓ memory segmentation (four 64 KB segments)
- ✓ 256 vector interrupts; minimum and maximum modes
- ✓ works with the 8087 coprocessor (ESC)

**Q10** From a given 8086 architecture figure, name the marked parts (ALU, BIU, EU, internal bus; Part-A = AX...DI, Part-B = CS/DS/SS/ES/IP, Part-C = ALU).

✎ **Asked in:** BGDCL AM(CSE) 2024 · BTCL JAM 2022

**Answer:** Part-A = **general-purpose, pointer and index registers** of the EU · Part-B = **segment registers + instruction pointer** of the BIU · Part-C = **ALU**. Name the unit (BIU/EU) too.

## → Registers: full names and count

| Group | Registers (full name) |
|---|---|
| General (8 halves) | AX accumulator · BX base · CX count · DX data (AH, AL, … **DL = DX low byte**) |
| Pointer / index | **SP** stack pointer · **BP** base pointer · **SI** source index · DI destination index |
| Segment + IP | CS code · DS data · SS stack · ES extra · **IP** instruction pointer |
| Flags | one 16-bit register, 9 flags used |

✎ **Asked in:** full names of SP, BP, SI, DL, IP: 46th BCS ICT · 16-bit registers = 4 + 4 + 4 + IP + flags = **14** ✎ **Asked in:** DoICT ANE 2019 · 8086 is a **16-bit** processor ✎ **Asked in:** Combined Bank Officer(IT) 2026 · address bus = **20-bit** ✎ **Asked in:** DoICT AP 2020 · 49th BCS CSE (2025)

# 6 Segmentation and the 20-bit physical address

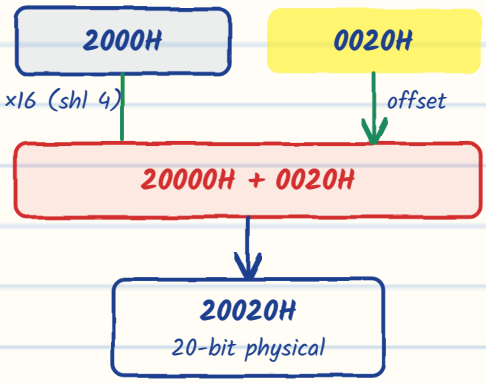
BCS every year

**Q11** Why is memory segmentation needed in the 8086? What is the technique to generate a 20-bit physical address in a 16-bit microprocessor? 5 marks

✎ **Asked in:** 45th BCS CSE (2024)

**Why:** registers are 16-bit (reach only 64 KB) but the address bus is 20-bit (1 MB). Segments let 16-bit registers reach 1 MB; they also separate code, data and stack, make programs relocatable and allow 64 KB blocks to overlap.

**How:** ~~PA = segment × 10H + offset~~; shift the 16-bit segment left by 4 bits (append a hex 0) and add the 16-bit offset in the BIU's adder.




CS = 1234H, IP = 0022H → 12340H + 0022H = 12362H

| MCQ asked | Key | Asked in |
|---|---|---|
| Segment 2000H, offset 0020H → physical address | 20020H | ✎ Asked in: 49th BCS ICT (2025) |
| 2000:1000 → actual location | 21000H | ✎ Asked in: BUET M.Sc. April 2021 |
| After RESET, 8086 starts from | FFFF0H | ✎ Asked in: 49th BCS ICT (2025) |

**Exam trap** Reset sets **CS = FFFFH, IP = 0000H** → FFFF0H, 16 bytes below the top of memory (a far JMP sits there). A copy of this MCQ offers only FFFFFH / FFFFH / 00000H / 00001H and explains "FFFFF0H": that is 24 bits, impossible. Write FFFF0H and note the misprint.

**Q12** Discuss the roles of the segment registers and offset registers of the 8086 with examples.

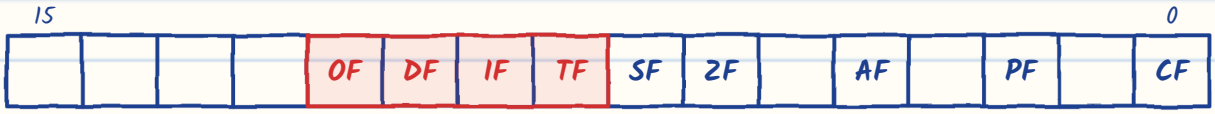
✎ **Asked in:** 47th BCS CSE

| Segment | Default offset register | Example |
|---|---|---|
| CS (code) | IP | next instruction at CS:IP |
| DS (data) | BX, SI, DI, direct address | MOV AL,[BX] reads DS:BX |
| SS (stack) | SP, BP | PUSH AX writes SS:SP−2 |
| ES (extra) | DI in string instructions | MOVSB copies DS:SI → ES:DI |

## 7 The flag register 5 exams

**Q13** What is the flag register of the 8086? Write its status and control flags. Discuss how flags are affected for signed and unsigned overflow.

✎ **Asked in:** 47th BCS CSE · DUET AP 2025 · DESCO SAE 2022 · BCPCL JI 2019




blank cells unused · status: CF PF AF ZF SF OF · control: TF IF DF

| Flag | Set (=1) when |
|---|---|
| CF carry | carry/borrow out of the MSB: **unsigned** overflow |
| PF parity | low byte has an **even** number of 1s |
| AF auxiliary | carry from bit 3 to bit 4 (BCD) |
| ZF zero / SF sign | result = 0 / MSB of result = 1 |
| OF overflow | signed result out of range: **signed** overflow |
| TF / IF / DF | single-step · enable INTR · string auto-decrement |

7FH + 01H = 80H → CF = 0, **OF = 1** (127 + 1 ≠ −128)

FFH + 01H = 00H → **CF = 1**, OF = 0, ZF = 1

**Memory trick** **C**arry for **C**ounting numbers (unsigned), **O**verflow for **O**pposite signs (signed). OF = carry into MSB XOR carry out of MSB.

"State flag" is **not** an 8086 flag ✎ **Asked in:** Combined Bank AHE/AE(IT) 2021

**Q14** Write the meaning of this 8086 program status word: S = 0, Z = 0, AC = 1, P = 1, CY = 0. 5 marks

✎ **Asked in:** 45th BCS CSE (2024)

S = 0 → result **positive** (MSB 0) · Z = 0 → result **not zero** · AC = 1 → a carry went from bit 3 to bit 4 · P = 1 → **even** number of 1s · CY = 0 → no carry out of the MSB. Example: 08H + 09H = 11H gives exactly S0 Z0 AC1 P1 CY0.

**Instructions that touch flags:** STC / CLC / CMC set, clear, complement CF · STD / CLD set/clear DF · STI / CLI set/clear IF · PUSHF / POPF save and restore the whole register · LAHF / SAHF copy the low flag byte to and from AH. There is no instruction to set TF directly: push flags, change bit 8, pop.

## 8 Memory size, odd and even banks DESCO · 47th BCS

**Q15** What is the maximum physical memory of the 8085 and the 8086, and why?

✎ **Asked in:** DESCO SAE 2022

8085: 16 address lines → $2^{16}$ = **64 KB**. 8086: 20 lines → $2^{20}$ = **1 MB**. Capacity = $2^{\text{address lines}}$ bytes.

**Q16** How are byte and word data read from odd and even addresses of the 8086?

✎ **Asked in:** 47th BCS CSE

Memory is two 512 KB banks: even (D0–D7, selected by **A0 = 0**) and odd (D8–D15, selected by **BHE' = 0**).

| Access | BHE' | A0 | Bus cycles |
|---|---|---|---|
| Byte at even address (MOV AL, [2000H]) | 1 | 0 | 1, lower bank |
| Byte at odd address (MOV AL, [2001H]) | 0 | 1 | 1, upper bank |
| Word at even address (aligned) | 0 | 0 | 1, both banks |
| **Word at odd address (MOV AX, [2001H])** | 0 / 1 | 1 / 0 | **2 cycles**: odd byte, then even byte |

## 9 8085 vs 8086 4 exams

**Q17** Differentiate between the 8085 and 8086 microprocessors (8-bit vs 16-bit).

✎ **Asked in:** 45th BCS ICT (2024) · PGCB SAE 2021 · BUET M.Sc. April 2017

| Feature | 8085 | 8086 |
|---|---|---|
| Data bus / ALU | **8-bit** | 16-bit |
| Address bus / memory | 16-bit / 64 KB | 20-bit / 1 MB |
| Segmentation | no | yes (CS, DS, SS, ES) |
| Pipelining | no | 6-byte instruction queue |
| Flags / modes | 5 flags; one mode | 9 flags; minimum + maximum mode |
| Multiply / divide | no MUL/DIV | MUL, IMUL, DIV, IDIV |

**8085 architecture** ✎ **Asked in:** 18th NTRCA School-2 (2024): accumulator + ALU + flags (S, Z, AC, P, CY) · registers B, C, D, E, H, L (pairs BC, DE, HL) · PC, SP · instruction register + decoder · timing & control · interrupt control (TRAP, RST 7.5/6.5/5.5, INTR) · serial I/O (SID/SOD) · multiplexed AD0–AD7 + A8–A15. 8085 is an **8-bit** MP ✎ **Asked in:** 49th BCS CSE (2025)

**Q18** The pin diagram of a microprocessor is given: name it, with reasons.

✎ **Asked in:** BUET M.Sc. October 2018

Read the pins: **AD0–AD15** + A16–A19/S3–S6 + **BHE'/S7** + MN/MX' on 40 pins → 8086 · AD0–AD7 + A8–A15 + **SID/SOD**, TRAP, RST 7.5/6.5/5.5, ALE → **8085** · AD0–AD7 + A8–A19 and SS0' in place of BHE' → **8088** (8-bit external bus). Give two pin reasons for full marks.

# 10 Addressing modes of the 8086

5 MPI + 3 CA exams

**Q19** What is an addressing mode and why is it needed? Describe immediate, direct, indirect, register and indexed addressing (any four) of the 8086 with your own examples.

✎ **Asked in:** 45th BCS CSE (2024) · 46th BCS CSE · 47th BCS ICT · BB AME 2017 · Combined Bank Officer(IT) 2025

An addressing mode is the way an instruction specifies where its operand is. Needed for flexibility (constants, variables, arrays, pointers, stack), shorter instructions and fewer registers wasted.

| Mode | Example | Operand / effective address (EA) |
|---|---|---|
| Immediate | `MOV AX, 0025H` | the data is inside the instruction |
| Register | `MOV AX, BX` | operand in a register (fastest, no memory access) |
| Direct | `MOV AX, [5000H]` | EA = 5000H (in DS) |
| Register indirect | `MOV AX, [BX]` | EA = BX (or SI, DI) |
| Based / register relative | `MOV CL, [BX+4]` | EA = BX or BP + displacement |
| **Indexed** | `MOV AL, [SI+10H]` | EA = SI or DI + displacement; arrays |
| Based-indexed (+ disp.) | `MOV AX, [BX+DI+4]` | EA = base + index (+ displacement); 2-D arrays |
| Relative (PC/IP) | `JNZ next` | target = IP + signed displacement |

**Exam trap** "the 12th microprocessor" in the Combined Bank 2025 paper is a misprint: answer for the 8086. Compare direct vs immediate vs register by **where the operand lives** (memory / instruction / register) and **memory accesses** (1 / 0 / 0).

**Q20** Write the addressing mode of: MOV AX, BX · MOV AX, [SI] · MOV AX, 006H · MOV [AX], [BX].

✎ **Asked in:** DESCO SAE(Computer) 2026

register · register indirect · immediate · **invalid**: the 8086 never allows memory-to-memory MOV, and AX cannot be used as a pointer (only BX, BP, SI, DI). Correct it as `MOV CX,[BX]` then `MOV [DI],CX`.

**Memory trick** "**I R D I B I**" in order of distance from the CPU: **I**mmediate (in the instruction), **R**egister, **D**irect, register **I**ndirect, **B**ased, **I**ndexed. Brackets [ ] = go to memory.

# 11 Assembly language and a first program

5 exams

**Q21** What is assembly language? Write its features. Write a program in assembly language to add and multiply two numbers. 5 marks

✎ **Asked in:** 45th BCS CSE (2024)

Assembly language is a low-level language that writes each machine instruction as a **mnemonic** (MOV, ADD) with symbolic operands; an **assembler** translates it 1 : 1 into machine code.

- ✓ machine-dependent (8086 code ≠ ARM code)
- ✓ direct access to registers, memory, I/O ports
- ✓ fast, compact code; used in drivers, boot code, embedded
- ✓ labels, directives (DB, DW), comments
- ✓ harder to write, debug and port than C

```
.MODEL SMALL
.DATA
  N1  DB 05H
  N2  DB 03H
  SUM DB ?
  PRD DW ?
.CODE
  MOV AX, @DATA
  MOV DS, AX      ; point DS at data
  MOV AL, N1
  ADD AL, N2      ; AL = 08H
  MOV SUM, AL
  MOV AL, N1
  MUL N2          ; AX = AL × N2 = 000FH
  MOV PRD, AX
  MOV AH, 4CH
  INT 21H         ; return to DOS
```

→ SUM = 08H (5 + 3) · PRD = 000FH (5 × 3). MUL with a byte operand multiplies by AL and leaves the 16-bit product in AX.

**Q22** Write the work of the instructions ADC, XCHG, POP, JNZ.

✎ **Asked in:** NWPGCL JAM 2020

| Instruction | Work | Example |
|---|---|---|
| ADC | add with carry: dest = dest + src + CF | ADC DX, BX (high word of a 32-bit add) |
| XCHG | swap two operands (not two memory ones) | XCHG AX, BX |
| POP | dest = word at SS:SP, then SP = SP + 2 | POP CX |
| JNZ | jump if ZF = 0 (result not zero) | DEC CX / JNZ loop |

## 12 MCQ drill: addressing and assembly BUET · BCS · bank

| MCQ asked | Key | Asked in |
|---|---|---|
| Register relative addressing example | MOV CL, [BX+4] | ✎ Asked in: BUET M.Sc. April 2021 |
| Not an addressing mode: Immediate / Direct / Indirect / Sequential | Sequential | ✎ Asked in: 49th BCS CSE (2025) |
| EA = constant + contents of a register | Index mode | ✎ Asked in: Combined Bank AHE/AE(IT) 2021 |
| PC = 825, address field = 24, relative mode → EA | 850 | ✎ Asked in: BUET IICT April 2021 |
| 32-bit instruction, 4-bit opcode, direct jump, byte memory → highest target | $2^{28} - 1$ | ✎ Asked in: BUET M.Sc. April 2021 |

**Exam trap (PC-relative)** While the instruction is read the PC is incremented to **826**, so EA = 826 + 24 = **850**. 849 is the distractor for "PC not yet updated". Write the assumption in one line.

| MCQ asked | Key | Asked in |
|---|---|---|
| `MOV AX, BX` | copies BX into AX | ✎ Asked in: 49th BCS ICT (2025) |
| `MOV BL,8C` · `MOV AL,7E` · `ADD AL,BL` → AL? | 0A, carry set | ✎ Asked in: BUET M.Sc. April 2021 |
| `mov ax,0` · `mov cx,0Ah` · `doloop: dec ax` · `loop doloop` | ax = FFF6h, cx = 0 | ✎ Asked in: Combined Bank SO(IT) 2021 |
| Passes an instruction from the 8086 to the 8087 | ESC | ✎ Asked in: 49th BCS ICT (2025) |

Checked: 7EH + 8CH = 10AH → AL = 0AH, CF = 1 · LOOP decrements CX and repeats while CX ≠ 0: 10 passes, AX = 0 − 10 = FFF6H.

**Exam trap** In the ADD question the destination is **AL** (MOV AL comes second). In the loop question, LOOP itself decrements CX: do not count an extra pass.

**Q23** Using only `mov a,b` · `add a,b` · `add a,k`, write PUSH and POP. Given `beq a,b,c`-style code, what are its limitations?
✎ Asked in: BUET M.Sc. May 2019

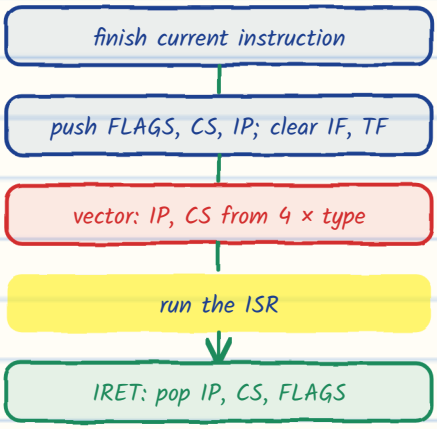
```
PUSH x:  add sp, -4     ; sp = sp - 4
         mov [sp], x
POP  x:  mov x, [sp]
         add sp, 4
```

Assumes `mov` can address memory through a register and `k` is a signed constant. Limits of a tiny ISA: no subtract or compare (use add of a negative), every PUSH/POP costs 2 instructions, a branch needs 3 operands, no overflow checks.

## 13 Interrupts BCS written

**Q24** Define interrupt with an example. Write the types of software interrupts. How is an interrupt enabled, and what are the steps when an interrupt is processed?
✎ **Asked in:** 45th BCS CSE (2024) · 38th BCS (2018)

An interrupt is a signal that makes the CPU pause the current program, run an **interrupt service routine (ISR)**, then resume. Example: a key press interrupts the CPU so it reads the keyboard.




**Software interrupts** (8086): `INT n` (n = 0–255, e.g. INT 21H DOS service) · `INT 3` breakpoint · `INTO` on overflow · exceptions: divide error (type 0), single-step (type 1).
**Enable:** `STI` sets IF = 1 (maskable INTR accepted), `CLI` clears it; unmask the line in the 8259 PIC. NMI cannot be masked.
**Interrupt controller** ✎ **Asked in:** 36th BCS (2016): the 8259A PIC takes 8 interrupt requests (cascade to 64), resolves priority, masks lines and gives the CPU the vector type. Hardware interrupt pins of the 8086: **INTR and NMI** ✎ **Asked in:** BUET M.Sc. April 2021 · interrupt-driven I/O beats polling because it **reduces CPU idle time** ✎ **Asked in:** 49th BCS CSE (2025)

## 14 DMA and the 8237 controller 47th BCS · BCPCL

**Q25** How does DMA increase system concurrency? List the data transfer modes of the 8237 DMA controller with their specialty. (What does a DMA controller do?)
✎ **Asked in:** 47th BCS ICT · BCPCL JI 2019

DMA lets an I/O device move a block directly to or from memory. The CPU only sets address, count and direction in the DMAC; the DMAC raises HOLD, gets HLDA, drives the bus and interrupts at the end. The CPU keeps computing (from cache/queue) meanwhile, so I/O and computation **overlap**.

| 8237 mode | Specialty |
|---|---|
| Single | one byte per request; bus returned each time |
| Block | whole block in one go (fastest) |
| Demand | while DREQ stays active |
| Cascade | chains another 8237 |

## 15 80386, Pentium and superscalar

BUET favourite

**Q26** What are the advantages of the 80386 over the 80286?

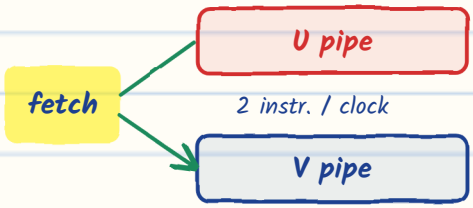
✎ **Asked in:** BUET M.Sc. October 2017

80386: **32-bit** registers and buses (80286: 16-bit) · 32-bit address → **4 GB** (80286: 24-bit, 16 MB) · **paging** with segmentation (virtual memory) · **virtual-8086 mode** to run DOS programs in protected mode · switches back to real mode without reset · 3-stage pipeline.

**Q27** What is a superscalar processor? Why is the Pentium superscalar but the 80386 not?

✎ **Asked in:** BUET M.Sc. October 2017 · BUET M.Sc. April 2017 · BUET M.Sc. May 2016 · BUET M.Sc. October 2015 · MM AP 2017

A superscalar CPU has several execution pipelines and issues **more than one instruction per clock** (CPI < 1). The Pentium has two integer pipelines, **U and V**, that run two simple independent instructions together, plus a separate FPU. The 80386 has one pipeline, so at best one instruction per clock: pipelined, not superscalar.




## 16 Cores, threads, hyper-threading

bank & power papers

**Q28** What does "2 cores and 4 threads" mean? What is hyper-threading and its use? Difference between Core i3, i5 and i7 (hardware).

✎ **Asked in:** BTRC AD 2021 · BOF AP 2022 · DESCO SAE 2022 · Combined Bank SO(IT) 2023

**Core** = a complete execution unit; **thread** = an instruction stream the OS can schedule. 2 cores / 4 threads = each core runs 2 threads by hyper-threading (Intel SMT): one physical core keeps two register sets and fills idle execution units from the second thread (≈ 15–30 % more throughput, not 2×). **i3** → **i5** → **i7** → **i9**: more cores/threads, larger L3 cache, higher turbo clock, more features; compare within the same generation.

| MCQ asked | Key | Asked in |
|---|---|---|
| Hyper-threading … | **one physical core runs several threads** | ✎ **Asked in:** 49th BCS CSE (2025) |
| Cores / threads of the Core i7-9700K | **8 / 8** | ✎ **Asked in:** Combined Bank SO(IT) 2021 |

**Exam trap** The i7-9700K is the one 9th-gen i7 **without** hyper-threading: 8 cores, 8 threads. "i7 = always 2 threads per core" is wrong.

## 17 8255 modes, DMA keys, memory-mapped I/O

49th BCS · BTRC · bank

**8255 modes:** Mode 0 simple I/O · **Mode 1 strobed (handshake) I/O** · Mode 2 bidirectional (port A only) · BSR mode sets/resets port C bits. Handshake MCQ → Mode 1 ✎ **Asked in:** 49th BCS ICT (2025)

DMA allows the CPU to execute while I/O transfers data to memory ✎ **Asked in:** 49th BCS CSE (2025) · DMA = Direct Memory Access ✎ **Asked in:** BUET IICT April 2021 · fastest transfer mode: DMA ✎ **Asked in:** BPSC AME 2022

### → Memory-mapped vs isolated I/O

Memory-mapped: ports share the memory address space, reached by ordinary memory instructions (MOV, LDA, STA). So in a memory-mapped system the instructions **not used** are **IN / OUT** ✎ **Asked in:** BTRC SAD 2021 · BTRC AD(Technical) 2019; the feature that does **not** apply: "new instructions are required to access device registers" ✎ **Asked in:** Combined Bank AHE/AE(IT) 2021

**Exam trap** Printed keys here say "LDA" (BTRC) and "address space is used up" (Combined Bank): both describe what memory-mapped I/O **does**. Answer IN/OUT and "new instructions required".

## 18 CPU vs GPU, Amdahl, parallel processing

2025–2026 papers

**Q29** Compare CPU and GPU. Why are heterogeneous multicore processors (HMP) popular?

✎ **Asked in:** MoFood Network/Website Manager 2025 · Combined Bank SO(IT) 2020

CPU: few powerful cores, low latency, complex control, serial tasks. GPU: thousands of simple cores, high throughput, data-parallel (graphics, AI). HMP (ARM big.LITTLE, Intel P/E cores): big cores for bursts, small cores for background → better performance per watt.

**Q30** Define Amdahl's law: how does it limit the speedup from adding cores?

✎ **Asked in:** RAKUB ANSE 2026

$S(n) = 1 / ((1 - p) + p/n)$, $p$ = parallel fraction. As $n \to \infty$, $S \to 1/(1 - p)$.

$p = 0.9$: $n = 4 \to 3.08\times$; $n = \infty \to 10\times$ at most

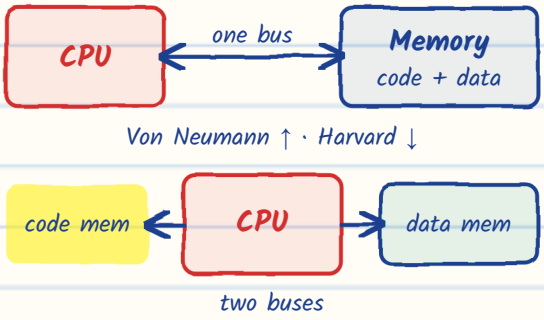
**Parallel processing**: several processing elements work on parts of a task at once (pipeline, vector/array, multiprocessor). **Multiprocessing**: two or more CPUs share memory and OS ✎ **Asked in:** 36th BCS (2016) · 18th NTRCA School-2 (2024) · 15th NTRCA CSE · **Array processor**: many identical PEs execute one instruction on many data items (SIMD), attached or embedded type ✎ **Asked in:** 15th NTRCA Computer Operation (2019)

## 19 Von Neumann vs Harvard

47th BCS · 49th BCS

**Q31** What are the differences between Harvard and Von Neumann architecture? Explain with a diagram.

✎ **Asked in:** 47th BCS ICT




| Von Neumann | Harvard |
|---|---|
| **same** memory + bus for code and data | separate memories and buses |
| fetch and data access take turns (bottleneck) | both at once: faster |
| simpler, cheaper, flexible | more wiring; fixed split |
| PCs (at main-memory level) | DSPs, microcontrollers, split L1 cache |

Von Neumann uses the **same** memory for data and instructions ✎ **Asked in:** 49th BCS ICT (2025) · "computer architecture with diagram" = CPU (ALU + CU + registers), memory, I/O joined by the system bus ✎ **Asked in:** Intl Crimes Tribunal Hardware Technician 2024 · internal blocks: 15th NTRCA Computer Operation (2019)

## 20 System bus, instruction cycle, registers

NTRCA written

**Q32** What is a system bus? Discuss its types with a diagram.

✎ **Asked in:** 16th NTRCA CSE (2019)

**Data bus** (bidirectional, width = word size) · **address bus** (unidirectional, CPU → memory/I/O, width sets capacity) · **control bus** (RD', WR', clock, interrupt, bus request).

**Q33** Describe the instruction fetch cycle and execution cycle.

✎ **Asked in:** 15th NTRCA CSE

Fetch: MAR ← PC; MBR ← M[MAR]; IR ← MBR; PC ← PC + 1. Decode: CU reads the opcode. Execute: fetch operands, ALU works, store result; check for interrupts.

**MAR** holds the address to be read/written; **MBR (MDR)** holds the data going to or coming from memory ✎ **Asked in:** 16th NTRCA CSE (2019) · **instruction set** = all machine instructions a CPU understands; types: data transfer, arithmetic, logical, branch/control, I/O, (string, shift) ✎ **Asked in:** 13th NTRCA CSE (2016)

## 21 RISC vs CISC

8 exams

**Q34** Mention / compare the differences between RISC and CISC processors (features and performance).

✎ **Asked in:** 46th BCS ICT · 47th BCS CSE · Combined Bank SO(IT) 2024 · 36th BCS (2016) · 16th NTRCA CSE (2019) · 13th NTRCA CSE (2016)

| Point | RISC (ARM, MIPS, RISC-V) | CISC (x86, VAX, 8086) |
|---|---|---|
| Instructions | few, simple | many, complex (string, MUL from memory) |
| Instruction length | **fixed** (32 bits) | variable (1–15 bytes on x86) |
| Cycles per instruction | ≈ 1 | several |
| Memory access | **load/store** only; ALU works on registers | ALU instructions can use memory |
| Addressing modes | few | many |
| Registers | many (32+) | fewer (8 on 8086) |
| Control unit | hardwired | microprogrammed |
| Pipelining | easy, efficient | harder (variable length decode) |
| Code size / compiler | larger code, smart compiler | compact code (memory-efficient) |

**Memory trick** **RISC = "Really Is Simple, Compiler does the work"**; CISC puts the complexity in hardware (microcode). Performance equation: time = instructions × CPI × clock period. RISC raises instruction count but cuts CPI.

**Q35** Fill in RISC or CISC: (i) pipelining less efficient due to instruction complexity · (ii) emphasis on hardware simplicity · (iii) complex decoding due to variable length · (iv) easy to pipeline · (v) more addressing modes · (vi) fixed instruction format · (vii) more memory efficient.

✎ **Asked in:** Titas Gas AE(CSE) 2024

→ (i) CISC · (ii) RISC · (iii) CISC · (iv) RISC · (v) CISC · (vi) RISC · (vii) CISC

**Exam trap** Modern x86 chips decode CISC instructions into RISC-like **micro-ops** internally; say so for full marks. "RISC and RISC" in the Combined Bank 2024 paper is a misprint for RISC and CISC.

## 22 Registers and micro-instructions: short questions

MCQ + match

| MCQ / match asked | Key | Asked in |
|---|---|---|
| Register pointing to the next instruction | **PC** | ✎ **Asked in:** 49th BCS CSE (2025) |
| Loaded with the memory word pointed to by PC | **Instruction register** | ✎ **Asked in:** Combined Bank AHE/AE(IT) 2021 |
| Sequence control register is also called | **Program counter** | ✎ **Asked in:** DoICT AP 2020 |
| Data bus / next-address reg. / ALU reg. / HW–SW link / board | **data bus · PC · accumulator · OS · motherboard** | ✎ **Asked in:** BSCPLC AM(Engineering) 2024 |
| Not a main CPU component: CU / ALU / cache / registers | **Cache** | ✎ **Asked in:** 49th BCS ICT (2025) |

**Q36** Main difference between vertical and horizontal micro-instructions?

✎ **Asked in:** Teletalk Executive 2019

Horizontal: one bit per control signal, long words, much parallelism, no decoding. Vertical: encoded fields, short words, needs a decoder, less parallel.

**Q37** Meaning and use of three-operand operations; which data structure suits them?

✎ **Asked in:** BUET M.Sc. October 2017

`ADD R1, R2, R3` (*R1 = R2 + R3*): sources kept, fewer moves, fits RISC. Expression evaluation uses a **tree / three-address code**; zero-address machines use a **stack**.

## 23 RISC and CISC in practice

BUET · Combined Bank 2026

**Q38** A processor's flow diagram is given: CISC or RISC? Explain in one sentence.

✎ **Asked in:** BUET M.Sc. October 2018

Look for **load/store** and one-cycle stages (IF ID EX MEM WB) → RISC; a microcode ROM / micro-sequencer in the control path → CISC.

**Q39** ARM (RISC) or x86 (CISC) for a new cloud data centre: which is better and why?

✎ **Asked in:** Combined Bank ADA 2026

ARM: better **performance per watt**, more cores per socket, lower cooling and cost (AWS Graviton) → good for scale-out web/microservices. x86: wider software compatibility, strong single-thread speed, legacy and licensed apps. Typical answer: ARM for new cloud-native fleets, keep x86 for legacy workloads.

# 24 CPU time, CPI and speedup

14 exams

**Formulas** CPU time = IC × CPI × T = IC × CPI / f · average CPI = Σ (fraction × cycles) · speedup = old time / new time · MIPS = f / (CPI × $10^6$)

| Question (as asked) | Answer | Asked in |
|---|---|---|
| $10^9$ instructions, CPI = 2, 1 GHz → execution time? | 2 s | ✎ Asked in: 49th BCS ICT (2025) |
| Mix: loads 25 % × 5, stores 10 % × 4, ALU 52 % × 4, branches 11 % × 3, jumps 2 % × 3 → CPI | 4.12 | ✎ Asked in: BUET M.Sc. April 2021 |
| Multi-cycle mix: memory 30 % × 6, ALU 60 % × 4, jumps 10 % × 2. How much faster than single-cycle? | 1.36 × | ✎ Asked in: BUET M.Sc. April 2021 |

CPI = 1.25 + 0.40 + 2.08 + 0.33 + 0.06 = 4.12 · multi-cycle CPI = 1.8 + 2.4 + 0.2 = 4.4 steps; single-cycle needs the longest instruction (6 steps) every time → 6 / 4.4 = 1.36

**Q40** Computer A has a 2 GHz processor and takes 250 ps per instruction; computer B has 2.5 GHz and 500 ps per instruction. Which is faster?
✎ **Asked in:** Jamuna Oil JO(IT & MIS) 2025

Time per instruction already includes CPI: A = 250 ps, B = 500 ps → **A is 2× faster** despite the lower clock. (A: CPI = 250 ps × 2 GHz = 0.5; B: CPI = 500 ps × 2.5 GHz = 1.25.) → A, by 500/250 = 2.0

**Exam trap** A higher GHz does **not** mean a faster computer: compare CPU time (IC × CPI × T), never clock rate alone.

**Q41** A server thread needs 25 ms per request when the data is in the block cache; otherwise a disk access adds 75 ms during which the thread sleeps. One-third of requests miss the cache. Requests/s with multithreading?
✎ **Asked in:** BUET M.Sc. April 2021

While one thread sleeps on the disk another uses the CPU, so the CPU is the only limit: 25 ms each → 1000 / 25 = 40 requests/s (option d). Single-threaded: 25 + ⅓ × 75 = 50 ms → 20/s. 66.67 is the textbook answer for 15 ms, a distractor.

**Q42** What is meant by latency, throughput and availability of a system?
✎ **Asked in:** 45th BCS ICT (2024)
**Latency**: time for one operation (response time, ns/ms). **Throughput**: operations completed per unit time (jobs/s, MB/s). **Availability**: fraction of time up = MTTF / (MTTF + MTTR), e.g. 99.99 %.

**Q43** Discuss the factors that affect CPU speed.
✎ **Asked in:** Rupali Bank ANA 2023 · "not a factor" MCQ: Combined Bank Officer(IT) 2020
Clock rate, CPI/IPC (microarchitecture), cores and threads, cache size and levels, word length, bus width and speed, RAM speed, pipeline depth, heat/throttling. Not a factor: e.g. hard-disk capacity or monitor size.

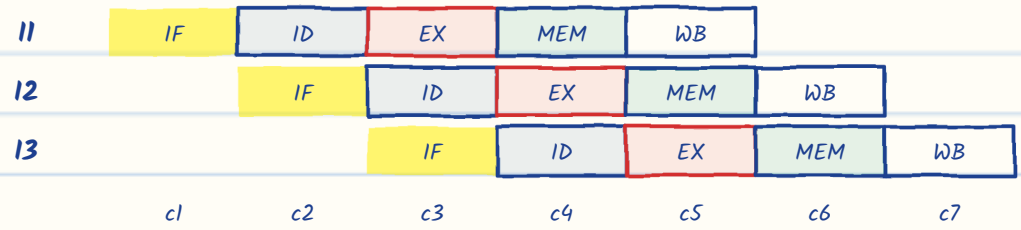
# 25 Pipelining

17 exams

**Q44** What is pipelining? How does it make instruction execution faster? Explain the stages (4-stage / 5-stage, DLX).

✎ **Asked in:** 46th BCS ICT · 38th BCS (2018) · Combined Bank AME/AE(IT) 2024 · BB AME 2019 · BTCL AE(CSE) 2017

**Pipelining** splits instruction execution into stages and overlaps them, like an assembly line: while one instruction executes, the next is decoded and a third is fetched. Each instruction still takes k cycles (latency unchanged) but one finishes **every cycle** (throughput up). 4-stage: Fetch, Decode, Execute, Write-back. **DLX / MIPS 5-stage:** **IF · ID · EX · MEM · WB**.




3 instructions in 7 cycles instead of 15

**Q45** Using a pipeline, calculate the fetch and execution cycles (time for n instructions).

✎ **Asked in:** BTCL AE(CSE) 2017

k stages, n instructions: pipelined = **k + (n − 1)** cycles; non-pipelined = n × k. Speedup = nk / (k + n − 1) → k for large n.

4 stages, 10 instructions: 4 + 9 = 13 cycles vs 40 (3.08×) · 5 stages, 100 instructions: 104 vs 500 → 4.81×

Real speedup stays below k: the pipeline must **fill** (k − 1 cycles), stages are not perfectly balanced (the clock fits the slowest), pipeline registers add delay, and **hazards** insert stalls.

**Q46** Why do modern processors favour a multi-stage pipelined approach over a single-cycle implementation?

✎ **Asked in:** Combined Bank AME/AE(IT) 2026(Jan) · Combined Bank SO(IT) 2026

Single-cycle: the clock must fit the **slowest** instruction (load), so every instruction pays that time and most hardware idles. Pipeline: the clock fits one short stage → higher frequency; stages work in parallel → CPI ≈ 1 at a much shorter cycle; hardware is reused. Cost: hazards and pipeline registers.

## 26 Hazards

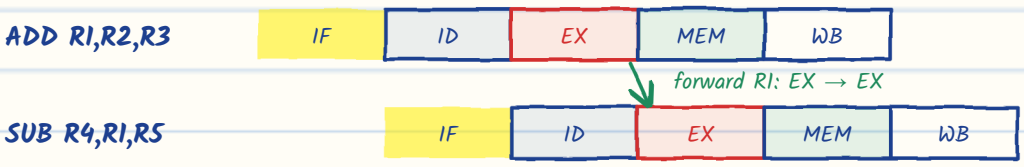
BUET · MoFood 2025

**Q47** Explain hazards in computer architecture. What is a data hazard? Why does branching pose a problem for pipelining, and how can it be tackled?

✎ **Asked in:** BUET M.Sc. April 2017 · BUET M.Sc. October 2018 · MoFood Network/Website Manager 2025

| Hazard | Cause / example | Fix |
|---|---|---|
| Structural | two stages need one resource (one memory port) | duplicate resources, split I/D cache |
| **Data** (RAW) | ADD R1,R2,R3 then SUB R4,R1,R5 | **forwarding**, stall, reorder |
| **Control (branch)** | target unknown until EX → wrongly fetched instructions | branch prediction, delayed branch, flush |

An instruction pipeline is built with a **FIFO** buffer ✎ **Asked in:** BUET IICT April 2021 · categories of architecture (Flynn): **SISD, SIMD, MISD, MIMD** ✎ **Asked in:** BB AME 2019




without forwarding, SUB must wait for ADD's write-back of R1: stall cycles

**Worked example: branch penalty** 20 % of instructions are branches; an unresolved branch costs 3 stall cycles. No prediction: CPI = 1 + 0.2 × 3 = **1.6**. A predictor that is right 90 % of the time: CPI = 1 + 0.2 × 0.1 × 3 = **1.06**. A delayed branch fills the slot after the branch with a useful instruction.

### ➜ Number representation asked with CA

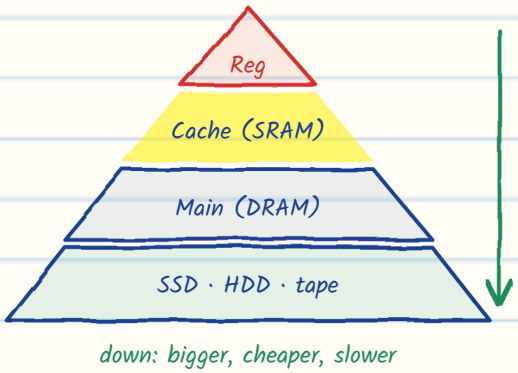
$(11111111)_2$ in an 8-bit register, 2's complement = **−1** ✎ **Asked in:** BPSC AME 2022 · machine epsilon = gap between 1 and the next larger representable float ($2^{-52}$ for double) ✎ **Asked in:** 49th BCS ICT (2025) · rounding error: a float has a fixed number of mantissa bits, so 0.1 is stored approximately; errors grow in long sums and in subtracting nearly equal numbers ✎ **Asked in:** 47th BCS CSE

## 27 Memory hierarchy and classification

24 exams

**Q48** What is computer memory? Classify it. Differentiate primary, secondary and tertiary memory (with examples). How does the hierarchy close the CPU–memory speed gap?

✎ **Asked in:** 18th NTRCA School (2024) · Parliament Secretariat SAE 2023 · 15th NTRCA Computer Operation (2019) · 16th NTRCA CSE (2019) · Combined Bank AME/AE(IT) 2026(Jan)




down: bigger, cheaper, slower

| Level | Examples | Note |
|---|---|---|
| Primary | registers, cache, RAM, ROM | CPU accesses directly; RAM volatile |
| Secondary | HDD, SSD, pen drive | non-volatile, via I/O |
| Tertiary | tape library, optical jukebox | archival, mounted on demand |

The gap: the CPU wants data in ≈ 0.3 ns, DRAM answers in ≈ 60–100 ns. The hierarchy keeps the **most-used** data in small fast levels (locality), so most accesses hit the cache and the average time stays near the cache's speed.

**Q49** What is memory organization? What are the main physical and operational differences between SRAM and DRAM? Why SRAM for cache and DRAM for main memory?

✎ **Asked in:** Combined Bank AME/AHE 2023 · BUET AP(Exam Controller) 2025 · BUET AP 2025 · BPSC Lecturer 2019 · NESCO AM(ICT) 2020

| Point | SRAM | DRAM |
|---|---|---|
| Cell | **6 transistors** (flip-flop) | **1 transistor + 1 capacitor** |
| Refresh | not needed | **needs refresh** (≈ every 64 ms) |
| Speed / cost per bit | fast (≈ 1–10 ns) / high | slower (≈ 50–100 ns) / low |
| Density / power per bit | low / higher | high / lower |
| Use | cache | main memory (DDR4, DDR5) |

**Memory organization** = how cells are arranged (words × bits, banks, chips, address decoding) and how levels are connected. DRAM is used as main memory because it is denser, cheaper and uses less power per bit; key "**consumes less power**" ✎ **Asked in:** BUET IICT April 2021; it is **not** faster.

## 28 Address lines and memory expansion

numericals

**Q50** Memory size is 32 MB. How many address bits are needed to address each byte?

✎ **Asked in:** W2PDCL SAE 2023

32 MB = $2^5 \times 2^{20} = 2^{25}$ bytes → 25 address lines. Rule: $n = \log_2$(number of locations).

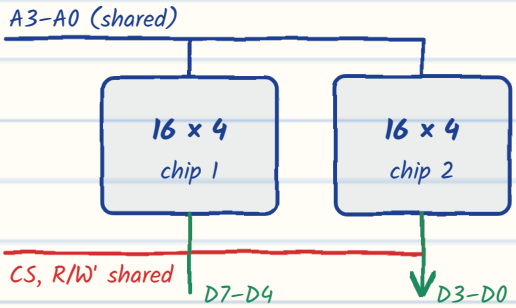
**Q51** Magnetic disk: 32 surfaces, 128 tracks/surface, 256 sectors/track, 1024 bytes/sector. Capacity?

✎ **Asked in:** Combined Bank AHE/AE(IT) 2021

$2^5 \times 2^7 \times 2^8 \times 2^{10} = 2^{30}$ bytes = 1 GB (option d)

**Q52** Draw a 16 × 8 RAM using 16 × 4 RAM chips.

✎ **Asked in:** Combined Bank SO 2020




Same number of words (16), wider word (8 bits): put the two chips **side by side**. Both get the same 4 address lines, chip select and R/W'; chip 1 gives the high nibble, chip 2 the low nibble. Chips needed = (16 × 8)/(16 × 4) = **2**. (More words instead would need a decoder on extra address lines.)

## 29 Upgrades and performance parameters

ICB 2026 · bank

**Q53** A fast CPU waits on slow RAM: upgrade the CPU or the RAM? If RAM speed is doubled, is latency halved?

✎ **Asked in:** ICB Securities AP 2026

Upgrade the **memory side** (faster RAM, dual-channel, more cache): a faster CPU would only wait more. Doubling the transfer rate (MT/s) raises **bandwidth**; latency (CAS cycles × cycle time) improves little, because CAS cycles usually rise too: DDR4-3200 CL16 = 10 ns, DDR5-6400 CL32 = 10 ns.

**Q54** Parameters used to evaluate memory performance. Effect of increasing RAM, CPU cache and bus width.

✎ **Asked in:** Combined Bank SO(IT) 2020 · JBL ANA 2022

**Access time**, **cycle time**, **bandwidth/transfer rate**, latency, capacity, cost per bit. More RAM → less paging; more cache → higher hit ratio; wider bus → more bytes per transfer.

## 30 RAM, ROM, BIOS, CMOS

short written + MCQ

**Q55** Uses of RAM and ROM; short notes on ROM, RAM and cache.

✎ **Asked in:** BOF SAE(Computer) 2021 · Intl Crimes Tribunal Hardware Technician 2024

RAM: read-write, volatile, holds the running OS, programs and data. ROM: read-only, non-volatile, holds firmware/boot code (PROM, EPROM, EEPROM, flash). Cache: small SRAM between CPU and RAM.

**Q56** What is BIOS? Difference between BIOS and CMOS.

✎ **Asked in:** BOF AE(CSE) 2021 · RAKUB ANSE 2023

**BIOS**: firmware in ROM/flash that runs POST, initialises hardware and loads the boot loader (UEFI today). **CMOS**: a small battery-backed RAM that stores BIOS settings (date, time, boot order). BIOS = program, CMOS = its settings.

BIOS / firmware is stored in **ROM (flash)** ✎ **Asked in:** Combined Bank Officer(IT) 2026 · BB AD(General) 2021 · volatile memory: **RAM** ✎ **Asked in:** BB AD(General) 2026 · non-volatile: ROM (HDD is also non-volatile; the MCQ is ambiguous) ✎ **Asked in:** BCC AP 2021 · not non-volatile: **LPDDR** (it is DRAM) ✎ **Asked in:** Combined Bank ADA 2021

Access time = average time to reach a location and get its contents ✎ **Asked in:** BUET IICT April 2021 · an address in main memory = **physical address** ✎ **Asked in:** BUET IICT April 2021 · virtual memory lives on the **hard disk** ✎ **Asked in:** DoICT ANE 2019 · LPDDR = **Low-Power Double Data Rate** → "none of the above" ✎ **Asked in:** Combined Bank SO(IT) 2021

## 31 SSD and fastest-device ordering

2025 papers

**Q57** Define SSD and its working principle; draw block and page in an SSD; SSD vs SATA (interface, speed, reliability).

✎ **Asked in:** MoFood Network/Website Manager 2025 · Combined Bank SO(IT) 2020 · BUET ICT Cell AHE 2025

SSD: NAND-flash storage with no moving parts; a controller maps logical blocks to flash pages (FTL), does wear-levelling and garbage collection. A **page** (4–16 KB) is the smallest unit read/written; a **block** (≈ 128–256 pages) is the smallest unit erased, so updates go to a fresh page. **SATA is an interface, not a drive type**: a SATA SSD tops out ≈ 550 MB/s; an NVMe SSD on PCIe reaches 3–7 GB/s; SSDs resist shock better than HDDs.

Fastest → slowest (ICT Ministry AP 2021): cache → RAM → SSD → flash drive → magnetic HD → DVD

✎ **Asked in:** ordering: ICT Ministry AP 2021 · fastest storage device: **SSD** ✎ **Asked in:** Combined Bank Officer(IT) 2026 · fastest memory: **cache** ✎ **Asked in:** DESCO AE(CSE) 2016

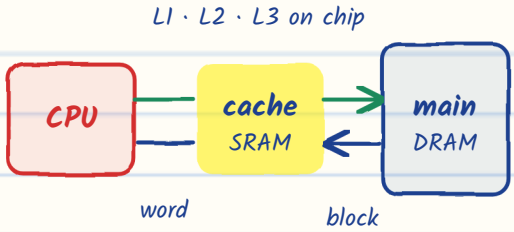
## 32 Cache memory and locality

53 exams ✦ the top topic

**Q58** What is cache memory? Explain locality of reference (spatial and temporal) with examples. Draw the organization of cache in a microprocessor system and explain how it speeds up execution.

✎ **Asked in:** 43rd BCS CSE · BPSC AP 2023 · 46th BCS CSE · Hydrocarbon Unit AD(ICT) 2025

Cache is a small, fast SRAM between the CPU and main memory that keeps copies of recently used blocks. The CPU checks the cache first: **hit** → data in a few ns; **miss** → fetch the whole block from RAM into the cache.




**Temporal locality**: a word used now will be used again soon (loop variable, loop code). **Spatial locality**: words near it will be used soon (array elements, next instructions). Locality is why a cache of a few MB serves a 16 GB RAM with 95 %+ hits.

Cache vs main memory: SRAM vs DRAM, KB–MB vs GB, ≈ 1–10 ns vs 60–100 ns, managed by hardware vs OS ✎ **Asked in:** BPSC ANE 2020 · cache vs virtual memory notes ✎ **Asked in:** MoFood Network/Website Manager 2025 · cache holds what the CPU is likely to **reuse** (true statement) ✎ **Asked in:** Combined Bank SO(IT) 2021 · registers are used to accept and transfer data the CPU is using **immediately** ✎ **Asked in:** BPSC AP 2022

## 33 Write-through vs write-back

4 exams

**Q59** Explain the differences between write-through and write-back cache write policies (merits and demerits). Which one causes more bus traffic?

✎ **Asked in:** Combined Bank SO(IT) 2020 · BTCL JAM 2018 · BUET M.Sc. October 2017

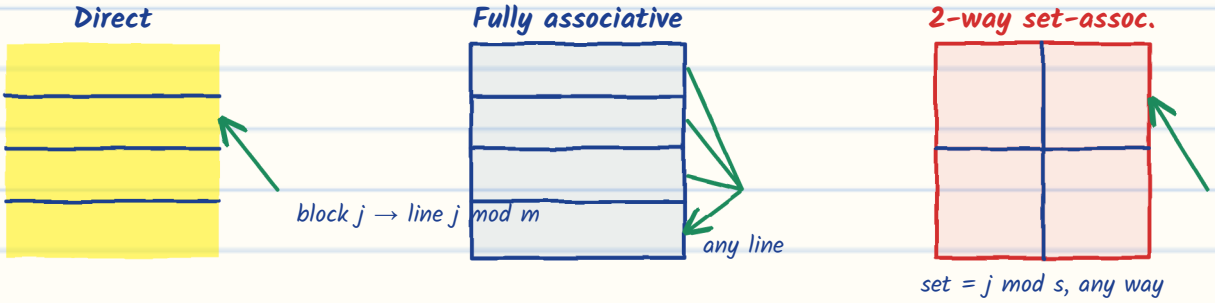
| | Write-through | Write-back |
|---|---|---|
| On a write hit | update cache **and** main memory | update cache only, set the dirty bit |
| Memory updated | immediately | when the dirty block is evicted |
| Bus traffic | **more** (every write) | less (one write per eviction) |
| Merit | memory always consistent; simple; good for I/O and multiprocessors | fast writes; saves bandwidth and power |
| Demerit | slow writes (needs a write buffer) | memory stale; complex; loss risk on failure |

## 34 Mapping: direct, fully associative, set-associative

BPSC AE 2022

**Q60** Differentiate direct, associative and set-associative mapping with diagrams; write the advantages and disadvantages of direct and associative mapping.

✎ **Asked in:** BPSC AE 2022 · DESCO SAE 2021 · BCIC AP 2025




| | Direct | Fully associative | k-way set-associative |
|---|---|---|---|
| Where block j goes | only line j mod lines | any line | any of k lines in set j mod sets |
| Address fields | tag · line · offset | tag · offset | tag · set · offset |
| Advantage | simple, cheap, fast (1 comparator) | no conflict misses, best hit ratio | balance (common: 4-, 8-way) |
| Disadvantage | conflict misses (thrashing) | a comparator per line: costly, slow | k comparators + replacement logic |

"Random" is a **replacement** policy, not a mapping technique ✎ **Asked in:** 49th BCS ICT (2025)

## 35 Cache misses: the 3 Cs

5 exams

**Q61** What is a cache miss? Explain compulsory, capacity and conflict misses with techniques to eliminate them. (Compulsory vs capacity miss.)

✎ **Asked in:** BPDB AE(CSE) 2023 · PSB ADA 2019 · Combined Bank SO(IT) 2026

| Miss | Cause | Reduce by |
|---|---|---|
| **Compulsory** (cold) | first reference to a block: it was never in the cache | larger blocks, prefetching |
| **Capacity** | the working set is bigger than the whole cache | bigger cache, better locality in code |
| **Conflict** | blocks map to the same line/set and evict each other, though space is free elsewhere | higher associativity, victim cache |

**Memory trick** **3 Cs: Cold, Capacity, Collision.** A fully associative cache has **no** conflict misses; an infinite cache has only cold misses.

## 36 Splitting the address BPSC · BUET · bank




**Q62** A byte-addressable system has a 32-bit physical address, a 64 KB cache and 16-byte blocks. With direct mapping, find the bits for tag, line index and block offset.
✎ **Asked in:** BPSC AP 2022 · BUET M.Sc. 2019

offset = $\log_2 16 = 4$ · lines = 64 KB / 16 B = 4096 = $2^{12}$ → index = 12 · tag = 32 − 12 − 4 = 16.
→ tag 16 | index 12 | offset 4

**Q63** A 4-way set-associative cache of 32 KB with 64-byte blocks; 32-bit address. Find (i) the number of sets, (ii) tag, set and offset sizes.
✎ **Asked in:** PGCB AE(CSE) 2023 · Combined Bank AP 2020

blocks = 32 KB / 64 B = 512 · sets = 512 / 4 = **128** → set = 7 · offset = 6 · tag = 32 − 7 − 6 = 19. → 128 sets; tag 19 | set 7 | offset 6

**Exam trap** For set-associative, the index counts **sets**, not lines: divide by the ways first. Doubling the ways adds 1 bit to the tag.

**Q64** A 16 KB direct-mapped cache with 4-word blocks on a 32-bit architecture (32-bit words): find tag, index and offset. How many total bits does the cache need?
✎ **Asked in:** BB AME 2023 · Combined Bank Officer(IT) 2024

1. block = 4 words × 4 B = 16 B → offset 4 (2 word + 2 byte bits); lines = 16 KB / 16 B = 1024 → index 10
2. byte addressing: tag = 32 − 10 − 4 = **18**; per line = 128 data + 18 tag + 1 valid = 147 bits
3. total = 1024 × 147 = 150 528 bits = **147 Kbits** (≈ 18.4 KB)

Word addressing: offset 2, tag 20 → 149 bits per line → 149 Kbits · with a dirty bit (byte addressing) → 148 Kbits

**Exam trap** The options say "147 KB", but the answer is **Kbits**. The BB AME 2023 wording "a direct-mapped cache with 4 blocks" means 4-**word** blocks; with literally 4 blocks of 4 KB: offset 12, index 2, tag 18.

| MCQ asked (16 KB data, 4-word blocks, 32-bit address and words) | Key | Asked in |
|---|---|---|
| Size, byte addressable, with a dirty bit | 148 K(bits) | ✎ **Asked in:** BUET M.Sc. April 2021 |
| Overhead, word addressable | 14.09 % | ✎ **Asked in:** BUET M.Sc. April 2021 |

Overhead = (tag + valid) / line = 21 / 149 = 14.09 % · the other options are 19/147, 20/148 and 22/150: one per assumption

## 37 AMAT: the most-asked CA numerical

17 exams

**Two conventions: say which one you use** **Hierarchical** (cache checked first, miss also pays the cache time): **AMAT = Tc + (1 − h) × Tm** = h·Tc + (1 − h)(Tc + Tm). **Simultaneous** (memory starts in parallel): AMAT = h·Tc + (1 − h)·Tm. If the paper gives "miss time" or "miss penalty" directly, use it as given. Two levels: AMAT = T1 + m1 × (T2 + m2 × Tm).

| Question (as asked) | Working | Answer | Asked in |
|---|---|---|---|
| L1 2 ns, hit 95 %, miss penalty 60 ns | 2 + 0.05 × 60 | **5 ns** | ✎ **Asked in:** 44th BCS CSE (2023) · BREB 2019 |
| Hit 90 %, Tc 10 ns, Tm 100 ns | 10 + 0.1 × 100 | **20 ns (19 simult.)** | ✎ **Asked in:** 46th BCS ICT |
| Tm 100 ns, Tc 50 ns, hit 90 % | 50 + 0.1 × 100 | **60 ns (55 simult.)** | ✎ **Asked in:** BB AME 2017 |
| Hit 80 %, Tc 5 ns, Tm 50 ns; options 15/45/50/55 | 5 + 0.2 × 50 | **15 ns** | ✎ **Asked in:** 49th BCS CSE (2025) |
| Hit takes 5 ns, a miss takes 50 ns, hit ratio 80 % | 0.8 × 5 + 0.2 × 50 | **14 ns** | ✎ **Asked in:** PGCB AE(CSE) 2021 |
| Tc 12 ns, Tm 1.5 µs, h = 0.98 | 12 + 0.02 × 1500 | **42 ns** | ✎ **Asked in:** Combined Bank SO(IT) 2021 |
| Hit 90 %, hit time 2 ns, miss penalty 20 ns | 2 + 0.1 × 20 | **4 ns** | ✎ **Asked in:** BCIC AP 2025 |
| L1 1 ns / 90 %, L2 5 ns / 80 %, memory 50 ns | 1 + 0.1 × (5 + 0.2 × 50) | **2.5 ns** | ✎ **Asked in:** Titas Gas AE(CSE) 2024 · SGFL AE(IT) 2021 |
| L1 1 ns / 90 %, L2 10 ns / 80 %, memory 80 ns | 1 + 0.1 × (10 + 0.2 × 80) | **3.6 ns** | ✎ **Asked in:** 47th BCS CSE |

Every row recomputed; the 12 ns case gives 41.76 ns in the simultaneous form, so 42 ns is the key either way.

**Exam trap** The 47th BCS paper prints the L1 hit ratio as "00%": read it as 90 % and say so. Convert units first (1.5 µs = 1500 ns). Miss **rate** = 1 − hit rate; never multiply the hit time by the miss rate in the hierarchical form.

Average access time rises when the **hit ratio decreases** ✎ **Asked in:** Combined Bank AHE/AE(IT) 2021 (printed key "reduction of miss penalty" is wrong: a smaller penalty lowers it).

**Why is cache a factor in system performance? Find the access time for a given hit percentage** ✎ **Asked in:** BTRC AD 2021 · BAERA Engineer(CSE) November 2021: CPU time = IC × ($CPI_{base}$ + memory stalls per instruction) × T, and stalls = accesses × miss rate × miss penalty; a higher hit ratio cuts both AMAT and stalls.

## 38 Replacement: LRU, FIFO, LFU

17th NTRCA 2023

**Q65** Explain the cache replacement algorithms LRU, FIFO and LFU.

**Asked in:** 17th NTRCA ICT (2023) · BBL AM(ICT) 2019

**FIFO**: evict the block loaded earliest · **LRU**: evict the block not used for the longest time (temporal locality) · **LFU**: evict the block used least often (ties: least recent) · Random: any block. **Trace**: 3 lines, fully associative, references 1 2 3 1 4 1 2 5 1 2:

| Ref | 1 | 2 | 3 | 1 | 4 | 1 | 2 | 5 | 1 | 2 | Hits |
|---|---|---|---|---|---|---|---|---|---|---|---|
| FIFO | M | M | M | H | M (−1) | M (−2) | M (−3) | M (−4) | H | H | 3 |
| LRU | M | M | M | H | M (−2) | H | M (−3) | M (−4) | H | H | 4 |
| LFU | M | M | M | H | M (−2) | H | M (−3) | M (−4) | H | H | 4 |

→ FIFO evicts the busy block 1 and pays for it; LRU and LFU keep it. (−n) = evicted.

## 39 Conflict misses and split caches

BPDB · BUET · PGCB

**Q66** A direct-mapped cache of 8 blocks (0–7), initially empty. Block sequence 0, 4, 8, 2, 0, 4, 8, 2: hits, misses and hit ratio?

**Asked in:** BPDB AE(CSE) 2024 · BUET M.Sc. 2016

| Block | 0 | 4 | 8 | 2 | 0 | 4 | 8 | 2 |
|---|---|---|---|---|---|---|---|---|
| Line (mod 8) | 0 | 4 | 0 | 2 | 0 | 4 | 0 | 2 |
| Result | M | M | M (−0) | M | M (−8) | H | M (−0) | H |

→ 2 hits, 6 misses, hit ratio 2/8 = 25 %. Blocks 0 and 8 fight over line 0: conflict misses; a 2-way cache would hit both.

**Q67** Hit time 1 ns, miss penalty 25 ns, instruction miss rate 1 %, data miss rate 5 %, and 15 % of accesses are data. Find the average access time.

**Asked in:** PGCB AE(CSE) 2024

1. Instruction AMAT = 1 + 0.01 × 25 = 1.25 ns (1)
2. Data AMAT = 1 + 0.05 × 25 = 2.25 ns (1)
3. Weighted: 0.85 × 1.25 + 0.15 × 2.25 = 1.40 ns (2)

**Exam trap** A circulated solution adds 1 % + 5 % and ignores the 15 % data share: that is wrong. Weight each stream by its share of accesses.

# Night before the exam

read twice

1. 8086: 16-bit data, **20-bit** address → 1 MB; 8085: 8-bit, 16-bit address → 64 KB
2. PA = segment × 10H + offset; reset → FFFF0H
3. 8086 = BIU (segments, IP, 6-byte queue) + EU (ALU, general regs, flags)
4. 14 registers; 9 flags: 6 status (CF PF AF ZF SF OF) + 3 control (TF IF DF)
5. CF = unsigned overflow, OF = signed overflow
6. Word at odd address → 2 bus cycles (BHE', A0)
7. No memory-to-memory MOV; only BX, BP, SI, DI as pointers
8. PC-relative EA = PC (already +1) + disp.
9. 8255 Mode 1 = handshake; 8259 = PIC; 8237 = DMAC; ESC → 8087
10. Memory-mapped I/O: no IN/OUT; isolated I/O needs them
11. Microcontroller = CPU + RAM + ROM + I/O on one chip
12. CPU time = IC × CPI × T; never compare GHz alone
13. Pipeline: $k + n - 1$ cycles; speedup → $k$
14. Hazards: structural, data (forwarding), control (prediction)
15. Superscalar = > 1 instruction per clock (Pentium U + V)
16. RISC: fixed length, load/store, 1 CPI, hardwired
17. Amdahl: $S = 1 / ((1 - p) + p/n) \leq 1/(1 - p)$
18. SRAM 6T, no refresh, cache · DRAM 1T1C, refresh, main
19. offset = $\log_2$ block, index = $\log_2$ lines (sets), tag = rest
20. AMAT = Tc + miss rate × Tm (state the convention)
21. 3 Cs: cold, capacity, conflict; write-back uses a dirty bit
22. Total cache bits: lines × (data + tag + valid [+ dirty])

**All the tricks** **Processor = brain, controller = brain + body** · **C**arry for **C**ounting, **O**verflow for **O**pposite signs · I R D I B I addressing order · **R**eally **I**s **S**imple, **C**ompiler does the work · **3 Cs: Cold, Capacity, Collision** · more cores ≠ proportional speed (Amdahl)

**Top traps** FFFF0H, not FFFFFH · "147 KB" is 147 **Kbits** · PC-relative 850, not 849 · i7-9700K = 8 / 8 · memory-mapped I/O drops IN/OUT · DRAM is not faster, it is denser · lower miss penalty lowers AMAT · weight split I/D caches by access share · multithreaded server: CPU time alone limits throughput

## → Numbers to have on your fingertips

| Quantity | Value | Quantity | Value |
|---|---|---|---|
| $2^{10}$ / $2^{16}$ / $2^{20}$ / $2^{32}$ | 1 K / 64 K / 1 M / 4 G | 1 GHz period | 1 ns (2.4 GHz → 0.417 ns) |
| 8086 queue / pins | 6 bytes / 40 | 8086 interrupt types | 256 (vector table 1 KB at 00000H) |
| DLX stages | IF ID EX MEM WB | Flynn classes | SISD SIMD MISD MIMD |

## ✓ Practice MCQs (asked ones first)

try first, then check

**1.** 8086 physical address for 2000H:0020H: ✎ 49th BCS ICT (2025)

(a) 200200H (b) 20020H × 16
(c) 20020H (d) 20020H + 0020H

**2.** Which 8255 mode allows handshake I/O? ✎ 49th BCS ICT (2025)

(a) Mode 0 (b) Mode 1
(c) Mode 2 (d) None

**3.** Not a useful cache mapping technique: ✎ 49th BCS ICT (2025)

(a) Direct (b) Set associative
(c) Fully associative (d) Random

**4.** Hit 80 %, cache 5 ns, memory 50 ns → effective access time: ✎ 49th BCS CSE (2025)

(a) 15 ns (b) 45 ns
(c) 50 ns (d) 55 ns

**5.** $10^9$ instructions, CPI 2, 1 GHz: ✎ 49th BCS ICT (2025)

(a) 0.5 s (b) 1 s
(c) 3 s (d) 2 s

**6.** Hardware interrupt pins of the 8086: ✎ BUET M.Sc. April 2021

(a) INT0 and INTR (b) NMI and INTS
(c) INTR and NMI (d) INTA and NMI

**7.** Instruction pipeline is implemented with a: ✎ BUET IICT April 2021

(a) LIFO buffer (b) FIFO buffer
(c) stack (d) none

**8.** TLB is used to: ✎ 49th BCS CSE (2025)

(a) store code (b) control transfers
(c) handle misses (d) translate addresses fast

**9.** Not a non-volatile device: ✎ Combined Bank ADA 2021

(a) LPDDR (b) SSD
(c) HDD (d) NVRAM

**10.** Total disk capacity: 32 surfaces × 128 tracks × 256 sectors × 1 KB: ✎ Combined Bank AHE/AE(IT) 2021

(a) $2^{12}$ B (b) $2^{13}$ B
(c) $2^{30}$ B (d) $2^{17}$ B

Answer key → 1-c · 2-b · 3-d · 4-a · 5-d · 6-c · 7-b · 8-d · 9-a · 10-c

**Written question** A byte-addressable machine has a 32-bit address, a 64 KB direct-mapped cache with 16-byte blocks, a 2 ns hit time, 95 % hits and a 60 ns miss penalty. (a) Split the address. (b) Find the AMAT. (c) Would 2-way set-associative change the tag? 10 marks

✎ **Asked in:** BPSC AP 2022 · 44th BCS CSE (2023) (combined from both)

1. (a) offset 4, lines 4096 → index 12, tag 16 — 4
2. (b) AMAT = 2 + 0.05 × 60 = 5 ns — 3
3. (c) 2-way: 2048 sets → set 11 bits, tag 17 (one more bit); fewer conflict misses — 3

### → Before the exam ✓

- ✓ 8086 block diagram from memory
- ✓ physical address + reset vector
- ✓ MP vs MC table (7 rows)
- ✓ flags with a worked addition
- ✓ tag / index / offset in 1 minute
- ✓ AMAT both conventions
- ✓ RISC vs CISC + pipeline hazards
- ✓ SRAM vs DRAM, write-through vs write-back